

Persecución de robots

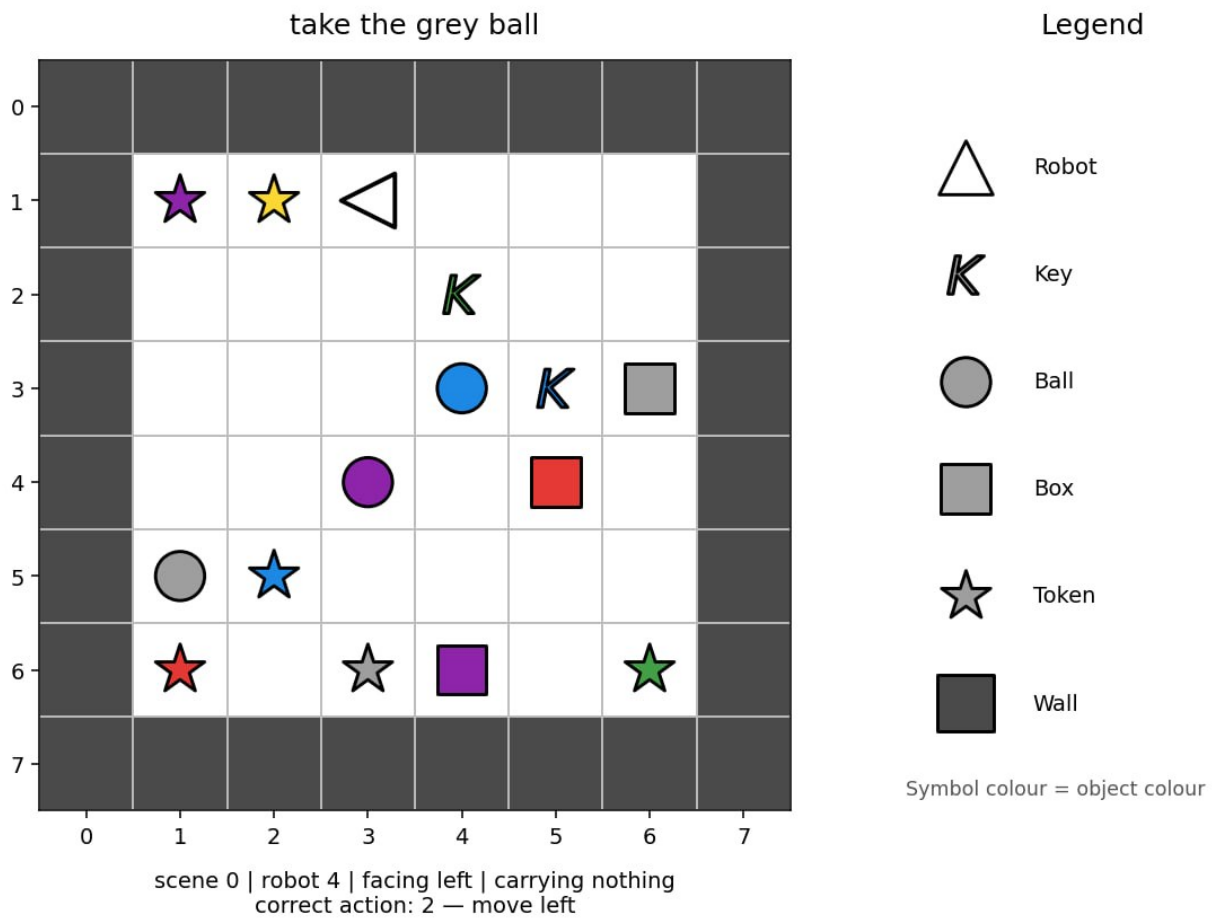
- **Límite de tiempo:** 5 minutos
- **Entorno:** una GPU (≈ 16 GB VRAM), sin internet
- **Tamaño de la solución:** `solution.ipynb` ≤ 1 MB
- **Almacenamiento:** 5 GB

Tarea

Hay seis robots. Cada robot opera en una habitación pequeña representada mediante una cuadrícula. Cada habitación tiene un área jugable de 6×6 rodeada de paredes, por lo que el array `image` completo tiene tamaño 8×8 (área jugable + paredes).

Cada robot recibe una instrucción en inglés que describe una tarea. Se toma una imagen instantánea en algún momento mientras el robot está llevando la instrucción a cabo. Tu objetivo es predecir la siguiente acción del robot.

Los robots no siempre siguen el camino más corto. El Robot 0 puede comportarse de manera diferente al Robot 1, pero cada robot sigue su propio patrón coherente. Utiliza los ejemplos de entrenamiento, que incluyen la siguiente acción correcta de cada robot, para aprender el patrón de cada uno.



Hay tres tipos de misiones:

- **ir hasta** un objeto, por ejemplo "approach the red ball";
- **recoger** un objeto, por ejemplo "grab the blue key";
- **colocar un objeto junto a otro**, por ejemplo "place the red box beside the green ball".

La misma instrucción puede escribirse de varias maneras. El conjunto de prueba puede contener nuevas combinaciones de frases, colores y tipos de objetos conocidos. Sin embargo, cada palabra, patrón de frase, color, tipo de objeto y tipo de misión que aparece en los datos de prueba también se usa en alguna muestra del conjunto de entrenamiento.

Cada muestra tiene los siguientes campos:

Campo	Significado
robot_id	cuál de los 6 robots es (0-5)
image	la habitación, un array de enteros $8 \times 8 \times 2$ en el que el canal 0 contiene el object_idx categórico (por ejemplo, 1=vacío, 2=pared, 10=robot. Puedes ver las opciones abajo) y el canal 1 contiene el colour_idx categórico (0-5).
direction	la dirección hacia la que está orientado actualmente el robot
mission	la instrucción visible en lenguaje natural
carrying	null o [object_idx, colour_idx] para el objeto transportado

Las filas son instantáneas independientes en orden aleatorio. No forman episodios, y durante la evaluación no tendrás ninguna observación ni acción anterior.

El `visualize_dataset.ipynb` proporcionado permite inspeccionar las observaciones disponibles para el modelo en diferentes situaciones.

Codificación de la cuadrícula

`image[row][column] = [object_idx, colour_idx]`. El primer índice es la fila, de arriba abajo, y el segundo es la columna, de izquierda a derecha. El array incluye el borde exterior de paredes, por lo que el interior transitable es 6×6 .

Ids posibles de objetos:

id	objeto
1	celda vacía
2	pared
5	llave
6	pelota
7	caja
10	robot
11	token

Aunque hay tokens en la habitación, estos nunca se los menciona en las misiones.

Los ids de color son 0 rojo, 1 verde, 2 azul, 3 morado, 4 amarillo y 5 gris. El canal de color no tiene significado para las celdas vacías ni para las paredes.

La imagen solo tiene los dos canales anteriores. La dirección del robot se proporciona una vez, en el campo de nivel superior `direction`; no está duplicada dentro de `image`.

Acciones

Para los códigos 0-3, las acciones de movimiento utilizan la siguiente correspondencia absoluta:

acción	significado
0	moverse hacia arriba
1	moverse hacia abajo
2	moverse hacia la izquierda
3	moverse hacia la derecha
4	recoger
5	soltar

El campo `direction` indica la orientación actual mediante: 0 = Arriba (fila - 1), 1 = Abajo (fila + 1), 2 = Izquierda (columna - 1), 3 = Derecha (columna + 1).

Una acción de movimiento primero gira el robot hacia esa dirección absoluta y después intenta desplazarlo una celda. Una pared o un objeto pueden bloquear el movimiento, pero la dirección cambia de todos modos. `pick up` y `drop` actúan sobre la celda adyacente definida por la dirección (por ejemplo, si `direction=0`, actúa sobre la celda en (fila - 1, columna), no sobre la celda del robot).

Dataset

Se proporcionan dos carpetas:

Carpeta	Filas	¿labels.json?	Utilízala para
<code>dataset/train/</code>	60,000	incluido	entrenar el modelo
<code>dataset/test_public/</code>	3,600	incluido en la copia de desarrollo	ejecutar y autoevaluar el pipeline

Cada carpeta contiene `observations.json`, una lista JSON de las muestras descritas anteriormente. `labels.json` es una lista JSON alineada de acciones (0-5).

El conjunto de entrenamiento contiene exactamente 10,000 filas por robot y 20,000 filas de cada tipo de tarea. El conjunto de prueba público contiene 600 filas por robot. Encierra `image` con `numpy.asarray(...)` si necesita un array.

En el momento de la calificación, `dataset/test_public/` se sustituye de forma transparente por un conjunto oculto de 3,600 observaciones con el mismo formato, pero sin `labels.json`. La tabla de clasificación pública utiliza `test_leaderboard_a`; la clasificación final utiliza `test_leaderboard_b`.

Un notebook que lea incondicionalmente las etiquetas `labels.json` de prueba fallará. Lee únicamente las etiquetas de entrenamiento de `dataset/train/`.

Salida

Escribe `predictions.json` en el directorio de trabajo del notebook. Debe ser una lista JSON que contenga una acción entera (0-5) por cada fila de `dataset/test_public/observations.json`, en el mismo orden. Para un conjunto de prueba hipotético que contenga seis muestras, una salida válida sería:

```
[0, 3, 2, 2, 5, 4]
```

Un archivo JSON ausente o no válido, un número incorrecto de predicciones, un valor no entero o una acción fuera de `{0,1,2,3,4,5}` se rechazan sin puntuación.

Puntuación

La puntuación es la **exactitud promedio por robot** en una escala de 0-100. Primero se calcula la exactitud de forma independiente para cada robot y después se promedia entre los seis robots. Por lo tanto, cada robot tiene el mismo peso.

Cómo enviar

1. Abre `solution.ipynb` y ejecute todas las celdas.
2. Confirma que escribe `predictions.json` con 3,600 predicciones para el conjunto de prueba público.
3. Mejora el modelo si lo desea; el baseline proporcionado solo muestra el formato requerido de entrada y salida.
4. En la pestaña Git de JupyterLab, cambia a stage y haz commit de `solution.ipynb` y, a continuación, haz push.
5. Vuelve a la página del concurso y haga clic en **Enviar**.

Envía exactamente un archivo llamado `solution.ipynb`.